

Diagnostic Imaging: Covid-19

COVID19 is an infectious disease that has changed the world as we know it and though vaccines have been created to reduce the rate of infection people are still coming down with the disease. Currently at home Covid-19 test kits are readily available and accessible to the general population but these kits still garner a significant amount of false negative results whether that be due to user error, the individual just having a low viral load in their nose despite having contracted the illness or the overall timing.

In the hospital, the nasal swab method is also used but all of the above issues are still possible, blood testing can be used to determine whether a patient has covid or not but this takes some time. Imaging provides faster results and with a model could potentially be even faster.

The Plan

To test this I will create a model that will utilize deep learning to look at a chest x-ray image and correctly determine whether the patient has Covid-19, Pneumonia, or nothing at all.

The chest x-rays utilized in this project were obtained from the COVID-QU-Ex Dataset on Kaggle which consists of 33,920 chest X-ray. The images in the set were collected and compiled by a team of researchers from the University of Dhaka and Qatar University.

LINK: <https://www.kaggle.com/datasets/anasmohammedtahir/covidqu>

Examples of images within the set:



✓ 1. Importing and Loading the data

```
# Importing and loading dataset
import zipfile
import os
import pandas as pd
from google.colab import files

uploaded = files.upload()

# define file paths
zip_path = "sample_600.zip"
images_dir = "images"
csv_output_path = "metadata.csv"

# create output directory
os.makedirs(images_dir, exist_ok=True)

# open the zip file
with zipfile.ZipFile(zip_path, 'r') as zip_ref:
    # loop through each file in the zip
    for file_name in zip_ref.namelist():

        if file_name.endswith('.png'):
            # extract image files to the images folder
            zip_ref.extract(file_name, images_dir)

        elif file_name.endswith('.csv'):
            # extract the metadata csv file separately
            zip_ref.extract(file_name, '.')
            os.rename(file_name, csv_output_path)

print("Extraction complete.")
print(f"Images saved in: {images_dir}/")
print(f"CSV file saved as: {csv_output_path}\n")

# view the metadata
print("\nMetadata:")
df = pd.read_csv("metadata.csv")
df
```

sample_600.zip

sample_600.zip(application/zip) - 17836115 bytes, last modified: n/a - 100% done
Saving sample_600.zip to sample_600.zip
Extraction complete.

Images saved in: images/
CSV file saved as: metadata.csv

Metadata:

	filename	label	
0	sub-S09492_ses-E18359_run-1_bp-chest_vp-ap_dx.png	covid	
1	sub-S09490_ses-E18365_run-1_bp-chest_vp-ap_dx.png	covid	
2	covid_2925.png	covid	
3	sub-S09382_ses-E20464_run-1_bp-chest_vp-ap_cr.png	covid	
4	sub-S09343_ses-E19246_run-1_bp-chest_vp-ap_dx.png	covid	
...	...	...	
595	Normal (9653).png	normal	
596	Normal (9233).png	normal	
597	Normal (527).png	normal	
598	Normal (8511).png	normal	
599	Normal (8305).png	normal	

600 rows × 2 columns

Next steps:

[Generate code with df](#)

[New interactive sheet](#)

```
# Display the amount of images in each class
df["label"].value_counts()
```

	count
label	
covid	200
pneumonia	200
normal	200

dtype: int64

In order to prevent class imbalancing 200 images from each class type were extracted. If were class were to have more images then the other then that class would be considered over represented and possible skew results



2. Stratified Split

```

import random
import numpy as np
from PIL import Image
from sklearn.model_selection import train_test_split
import torch

# set random seed so that results are reproducible
seed = 42
random.seed(seed)
np.random.seed(seed)
torch.manual_seed(seed)
torch.cuda.manual_seed_all(seed)

# Set device
device = torch.device("cuda" if torch.cuda.is_available() else "cpu")
print(f"Using device: {device}\n")

# Load metadata
print("Metadata columns:", list(df.columns))
print(f"Total samples: {len(df)}\n")

# Convert text labels to numeric labels
label_column = 'label'
classes = sorted(df[label_column].unique())
class_to_idx = {c: i for i, c in enumerate(classes)}
idx_to_class = {i: c for c, i in class_to_idx.items()}

# Create numeric label column
df['label_numeric'] = df[label_column].map(class_to_idx)

print(f"Classes: {classes}")
print(f"Class mapping: {class_to_idx}\n")

# Load images
images = []
for filename in df['filename']:
    # Search for the image file in subdirectories
    img_path = None
    for root, dirs, files in os.walk(images_dir):
        if filename in files:
            img_path = os.path.join(root, filename)
            break

    if img_path:
        img = Image.open(img_path)
        images.append(np.array(img))
    else:
        print(f"Warning: Image not found - {filename}")

images = np.array(images)
print(f"Images loaded: {images.shape}\n")

```

```

# Get metadata features (everything except labels and filenames)
metadata = df.drop(columns=['label', 'filename'])

# Create stratified split: 70% train, 15% val, 15% test
train_df, temp_df = train_test_split(
    df, test_size=0.30, stratify=df['label_numeric'], random_state=seed
)
val_df, test_df = train_test_split(
    temp_df, test_size=0.50, stratify=temp_df['label_numeric'], random_state=seed
)

print(f"Training: {len(train_df)} samples")
print(f"Validation: {len(val_df)} samples")
print(f"Testing: {len(test_df)} samples\n")

# Show label distribution
print("Label distribution:")
print(f"Train: {train_df['label_numeric'].value_counts().sort_index().to_dict()}")
print(f"Val:    {val_df['label_numeric'].value_counts().sort_index().to_dict()}")
print(f"Test:   {test_df['label_numeric'].value_counts().sort_index().to_dict()}")

```

Using device: cuda

Metadata columns: ['filename', 'label']

Total samples: 600

Classes: ['covid', 'normal', 'pneumonia']

Class mapping: {'covid': 0, 'normal': 1, 'pneumonia': 2}

Images loaded: (600, 256, 256)

Training: 420 samples

Validation: 90 samples

Testing: 90 samples

Label distribution:

Train: {0: 140, 1: 140, 2: 140}

Val: {0: 30, 1: 30, 2: 30}

Test: {0: 30, 1: 30, 2: 30}

I then performed a stratified split to separate the images into testing, training, and validation sets. The stratified split ensures that the same proportions are maintained in each set. Again we are trying to avoid class balance based errors in the model.

✓ 3. Setting Up The Model

In this project I utilize convolution neural networks specifically Resnet-50 as it is built to handle complex task such as this one. The model is pre-trained such that it just needs to adapt to the chest x-ray images rather than images as a whole.

```

from torch import nn
from torchvision import models
import torch.nn.functional as F

```

```
import torch.nn.functional as F
from torch.utils.data import Dataset, DataLoader
from torchvision import models, transforms
from torch.optim.lr_scheduler import CosineAnnealingLR
```

```
# Define Dataset class
```

```
class XRayDataset(Dataset):
    def __init__(self, df, img_dir, transform=None):
        self.df = df.reset_index(drop=True)
        self.img_dir = img_dir
        self.transform = transform

    def __len__(self):
        return len(self.df)

    def __getitem__(self, idx):
        row = self.df.iloc[idx]
        filename = row["filename"]

        # Find image in subdirectories
        img_path = None
        for root, dirs, files in os.walk(self.img_dir):
            if filename in files:
                img_path = os.path.join(root, filename)
                break

        img = Image.open(img_path).convert("L")
        if self.transform is not None:
            img = self.transform(img)
        label = int(row["label_numeric"])
        return img, label, filename
```

```
# Define transforms
```

```
train_tf = transforms.Compose([
    transforms.Grayscale(num_output_channels=3),
    transforms.Resize((400, 400)),
    transforms.RandomHorizontalFlip(p=0.5),
    transforms.ToTensor(),
    transforms.Normalize(mean=[0.485, 0.456, 0.406],
                          std=[0.229, 0.224, 0.225]),
])
```

```
eval_tf = transforms.Compose([
    transforms.Grayscale(num_output_channels=3),
    transforms.Resize((400, 400)),
    transforms.ToTensor(),
    transforms.Normalize(mean=[0.485, 0.456, 0.406],
                          std=[0.229, 0.224, 0.225]),
])
```

```
# Create datasets
```

```
train_ds = XRayDataset(train_df, images_dir, transform=train_tf)
val_ds = XRayDataset(val_df, images_dir, transform=eval_tf)
test_ds = XRayDataset(test_df, images_dir, transform=eval_tf)
```

```
# Create data loaders
```

```

batch_size = 8
num_workers = 4

train_loader = DataLoader(train_ds, batch_size=batch_size, shuffle=True, num_work
val_loader = DataLoader(val_ds, batch_size=batch_size, shuffle=False, num_workers
test_loader = DataLoader(test_ds, batch_size=batch_size, shuffle=False, num_worke

print(f"\nDataLoaders created:")
print(f"Train batches: {len(train_loader)}")
print(f"Val batches: {len(val_loader)}")
print(f"Test batches: {len(test_loader)}")

# Set up ResNet-50 model
num_classes = len(df['label'].unique())
classes = df['label'].unique()
print(f"\nNumber of classes: {num_classes}")
print(f"Classes: {classes}")
model = models.resnet50(pretrained=True)
model.fc = nn.Linear(model.fc.in_features, num_classes)
model = model.to(device)

print("ResNet-50 model loaded and configured")
print(f"Final layer: {model.fc}")

```

```

/usr/local/lib/python3.12/dist-packages/torch/utils/data/dataloader.py:627: UserWa
warnings.warn(
/usr/local/lib/python3.12/dist-packages/torchvision/models/_utils.py:208: UserWarr
warnings.warn(
/usr/local/lib/python3.12/dist-packages/torchvision/models/_utils.py:223: UserWarr
warnings.warn(msg)

DataLoaders created:
Train batches: 53
Val batches: 12
Test batches: 12

Number of classes: 3
Classes: ['covid' 'pneumonia' 'normal']
Downloading: "https://download.pytorch.org/models/resnet50-0676ba61.pth" to /root/
100%|██████████| 97.8M/97.8M [00:00<00:00, 166MB/s]
ResNet-50 model loaded and configured
Final layer: Linear(in_features=2048, out_features=3, bias=True)

```

4. Tuning and Training

Before train the model is fine tuned

After the tuning the training loop begins and the data for each epoch is stored to be analyzed later

```

# get the model weights for the pretrained ResNet-50 that will serve as our base
weights = models.ResNet50_Weights.IMAGENET1K_V2
model = models.resnet50(weights=weights)
model.fc = nn.Linear(model.fc.in_features, len(classes))
model = model.to(device)

```



```

model = model.to(device)

# Training setup
criterion = nn.CrossEntropyLoss()
optimizer = torch.optim.Adam(model.parameters(), lr=0.001, weight_decay = 3e-3)
num_epochs = 10
scheduler = CosineAnnealingLR(optimizer, T_max=num_epochs, eta_min=1e-7)

print(f"\nStarting training for {num_epochs} epochs...")

# Track metrics for plotting
train_losses = []
train_accs = []
val_losses = []
val_accs = []

# Training loop
for epoch in range(num_epochs):
    # Training phase
    model.train()
    train_loss = 0.0
    train_correct = 0
    train_total = 0

    for images, labels, _ in train_loader:
        images, labels = images.to(device), labels.to(device)

        optimizer.zero_grad()
        outputs = model(images)
        loss = criterion(outputs, labels)
        loss.backward()
        optimizer.step()

        train_loss += loss.item()
        _, predicted = torch.max(outputs.data, 1)
        train_total += labels.size(0)
        train_correct += (predicted == labels).sum().item()

    train_acc = 100 * train_correct / train_total
    avg_train_loss = train_loss / len(train_loader)

    # Validation phase
    model.eval()
    val_loss = 0.0
    val_correct = 0
    val_total = 0

    with torch.no_grad():
        for images, labels, _ in val_loader:
            images, labels = images.to(device), labels.to(device)
            outputs = model(images)
            loss = criterion(outputs, labels)

            val_loss += loss.item()
            _, predicted = torch.max(outputs.data, 1)
            val_total += labels.size(0)

```

```

        val_correct += (predicted == labels).sum().item()

    val_acc = 100 * val_correct / val_total
    avg_val_loss = val_loss / len(val_loader)

    # Save metrics
    train_losses.append(avg_train_loss)
    train_accs.append(train_acc)
    val_losses.append(avg_val_loss)
    val_accs.append(val_acc)

    # Update learning rate
    scheduler.step()

    print(f"Epoch [{epoch+1}/{num_epochs}] "
          f"Train Loss: {avg_train_loss:.4f}, Train Acc: {train_acc:.2f}% | "
          f"Val Loss: {avg_val_loss:.4f}, Val Acc: {val_acc:.2f}%")

print("\nTraining complete!")

```

```

Starting training for 10 epochs...
/usr/local/lib/python3.12/dist-packages/torch/utils/data/dataloader.py:627: UserWarning:
warnings.warn(
Epoch [1/10] Train Loss: 0.9217, Train Acc: 57.14% | Val Loss: 8.7208, Val Acc: 36.67%
Epoch [2/10] Train Loss: 0.8889, Train Acc: 60.71% | Val Loss: 4.3068, Val Acc: 43.33%
Epoch [3/10] Train Loss: 0.8286, Train Acc: 63.81% | Val Loss: 1.0376, Val Acc: 44.44%
Epoch [4/10] Train Loss: 0.7732, Train Acc: 69.29% | Val Loss: 1.3661, Val Acc: 50.00%
Epoch [5/10] Train Loss: 0.6763, Train Acc: 71.43% | Val Loss: 1.0061, Val Acc: 58.33%
Epoch [6/10] Train Loss: 0.6213, Train Acc: 77.86% | Val Loss: 2.6643, Val Acc: 52.78%
Epoch [7/10] Train Loss: 0.5215, Train Acc: 80.71% | Val Loss: 0.7088, Val Acc: 72.22%
Epoch [8/10] Train Loss: 0.3909, Train Acc: 86.90% | Val Loss: 0.5535, Val Acc: 81.11%
Epoch [9/10] Train Loss: 0.3131, Train Acc: 89.29% | Val Loss: 0.5428, Val Acc: 86.67%
Epoch [10/10] Train Loss: 0.2481, Train Acc: 92.62% | Val Loss: 0.5305, Val Acc: 86.67%

Training complete!

```

The initial training accuracy was 57.14% and the final training accuracy was 92.62%. The validation accuracy experienced a similar jump starting at 36.67% which was a little worrisome, and ending at 86.67%. From this I can assume that the model has trained really well and is ready for testing.

✓ 5. Plotting Curves

```

# Plot learning curves
import matplotlib.pyplot as plt

fig, (ax1, ax2) = plt.subplots(1, 2, figsize=(14, 5))

# Plot loss
ax1.plot(range(1, num_epochs+1), train_losses, 'b-', label='Train Loss')
ax1.plot(range(1, num_epochs+1), val_losses, 'r-', label='Val Loss')

```

```

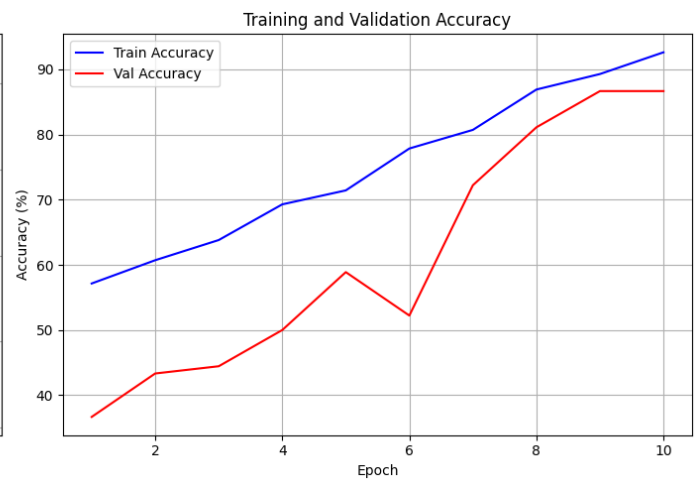
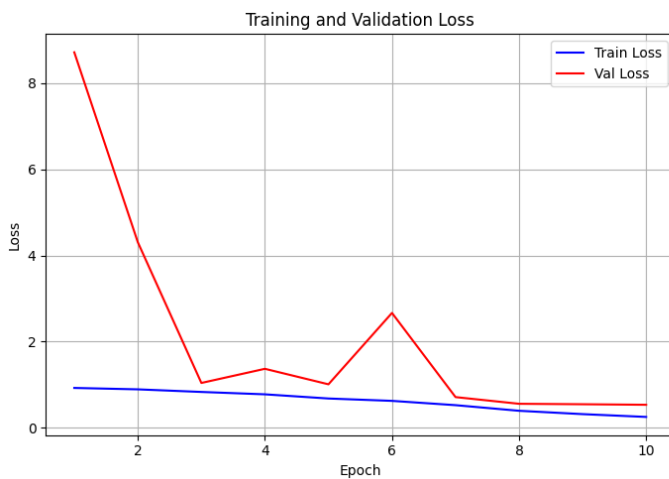
ax1.set_xlabel('Epoch')
ax1.set_ylabel('Loss')
ax1.set_title('Training and Validation Loss')
ax1.legend()
ax1.grid(True)

# Plot accuracy
ax2.plot(range(1, num_epochs+1), train_accs, 'b-', label='Train Accuracy')
ax2.plot(range(1, num_epochs+1), val_accs, 'r-', label='Val Accuracy')
ax2.set_xlabel('Epoch')
ax2.set_ylabel('Accuracy (%)')
ax2.set_title('Training and Validation Accuracy')
ax2.legend()
ax2.grid(True)

plt.tight_layout()
plt.show()

print(f"\nFinal Results:")
print(f"Train Accuracy: {train_accs[-1]:.2f}%")
print(f"Val Accuracy: {val_accs[-1]:.2f}%")

```



Final Results:
 Train Accuracy: 92.62%
 Val Accuracy: 86.67%

Looking at the loss curve for the validation there is initially high loss which means it was predicting all the wrong things but then the curves dives and follows the training set loss curve which is good

its likely that the learning rate was just high. The accuracy curve showcases a similar story but in both there is a strange spike in epoch 6 meaning the error in predictions randomly spiked. Overall its not too bad but the slight discrepancy between the training and validation set along with the spike make me believe the model is slightly overfit.

✓ 6. Testing the Model

```
from sklearn.metrics import confusion_matrix

# initialize variables
model.eval()
all_labels = []
all_preds = []

# run the model and generate a prediction for each image in the test set
with torch.no_grad():
    for imgs, labels, _ in test_loader:
        imgs = imgs.to(device)
        outputs = model(imgs)
        preds = outputs.argmax(dim=1).cpu().numpy()
        all_preds.append(preds)
        all_labels.append(labels.numpy())
all_preds = np.concatenate(all_preds)
all_labels = np.concatenate(all_labels)

# compute model accuracy on the test set
test_acc = (all_preds == all_labels).mean()

# generate a confusion matrix with results
cm = confusion_matrix(all_labels, all_preds, labels=list(range(len(classes))))
cm_norm = cm.astype(float) / cm.sum(axis=1, keepdims=True)

# calculate sensitivity and specificity for each class
sensitivities = []
specificities = []
for i in range(len(classes)):
    TP = cm[i, i]
    FN = cm[i, :].sum() - TP
    FP = cm[:, i].sum() - TP
    TN = cm.sum() - (TP + FN + FP)
    sens = TP / (TP + FN) if (TP + FN) > 0 else 0.0
    spec = TN / (TN + FP) if (TN + FP) > 0 else 0.0
    sensitivities.append(sens)
    specificities.append(spec)
macro_sens = np.mean(sensitivities)
macro_spec = np.mean(specificities)

# display accuracy, sensitivity, and specificity
print(f"\nOverall Accuracy: {test_acc*100:.2f}%\n")
for i, c in enumerate(classes):
    print(f"Class {c}: Sensitivity = {sensitivities[i]*100:.2f}% | Specificity =")
    print(f"\nOverall Sensitivity: {macro_sens*100:.2f}%")
```

```

print(f"\nOverall Sensitivity: {macro_sens*100:.2f}%")
print(f"Overall Specificity: {macro_spec*100:.2f}%\n")

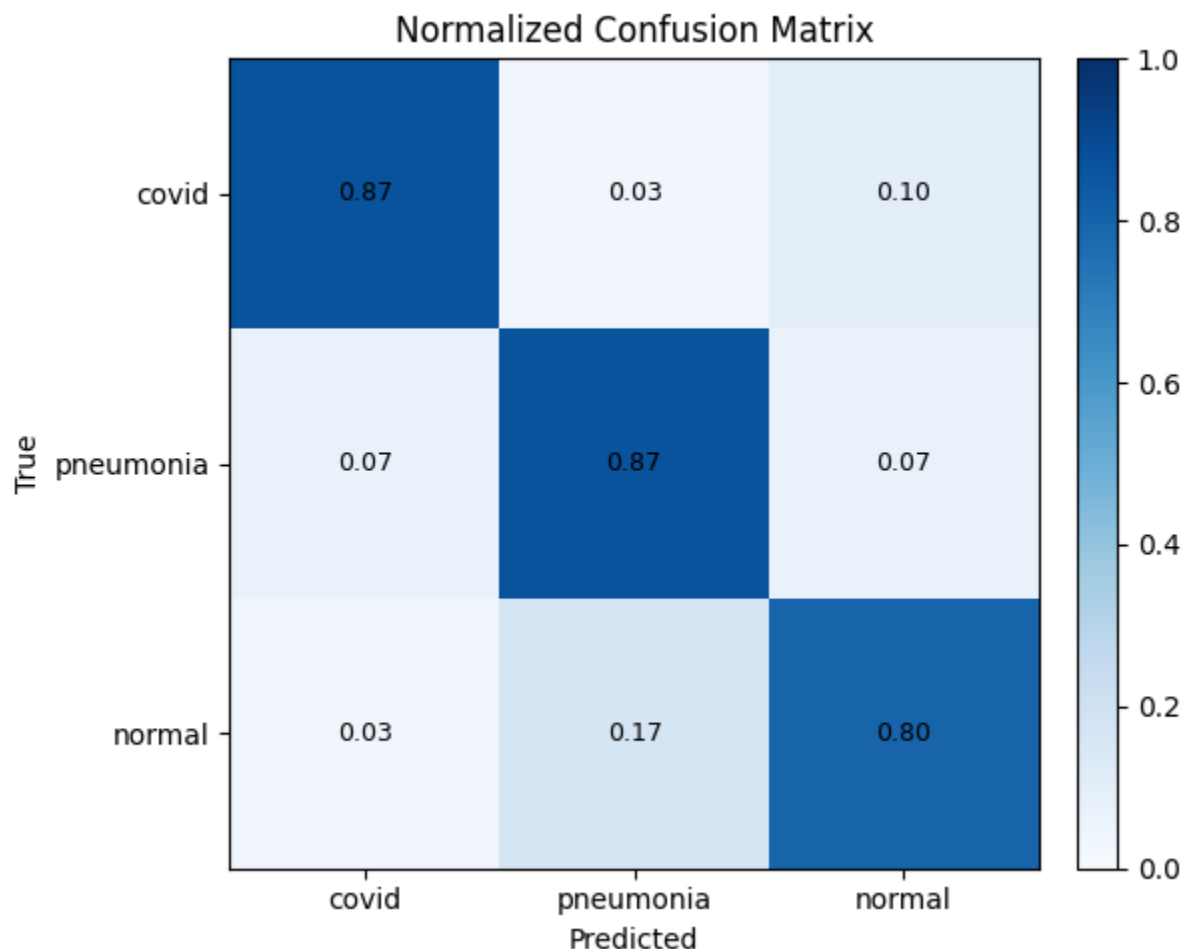
# plot the confusion matrix
fig, ax = plt.subplots(figsize=(6,5))
im = ax.imshow(cm_norm, cmap="Blues", vmin=0, vmax=1)
ax.set_xlabel("Predicted")
ax.set_ylabel("True")
ax.set_xticks(np.arange(len(classes))); ax.set_xticklabels(classes)
ax.set_yticks(np.arange(len(classes))); ax.set_yticklabels(classes)
plt.title("Normalized Confusion Matrix")
plt.colorbar(im, fraction=0.046, pad=0.04)
for i in range(len(classes)):
    for j in range(len(classes)):
        val = cm_norm[i, j]
        ax.text(j, i, f"{val:.2f}", ha="center", va="center", color="black", font
plt.tight_layout()
plt.show()

```

Overall Accuracy: 84.44%

Class covid: Sensitivity = 86.67% | Specificity = 95.00%
 Class pneumonia: Sensitivity = 86.67% | Specificity = 90.00%
 Class normal: Sensitivity = 80.00% | Specificity = 91.67%

Overall Sensitivity: 84.44%
 Overall Specificity: 92.22%



The overall accuracy of the model is 84.44% which is decent could be better. The model appears to be best at labeling covid chest x-rays and covid but not as good as correctly identifying a normal chest x-ray as normal but even at that its accuracy is still 80% for this class.

The model's specificity is 92.22% which is higher than its sensitivity and shows that the model is good at minimizing false positives and I feel in the context of trying to diagnose someone minimizing false positives is key in avoiding giving a patient the wrong treatment.

✓ 7. Grad Cam Visualizations

In order to truly determine the model's accuracy we have to see where and what the model is focusing on. The grad-cam provides heat-maps that depict exactly that.

```
import torch
import torch.nn.functional as F
import numpy as np
import matplotlib.pyplot as plt
from PIL import Image
import cv2

class GradCAM:
    def __init__(self, model, target_layer):
        self.model = model
        self.target_layer = target_layer
        self.gradients = None
        self.activations = None

        # Register hooks
        target_layer.register_forward_hook(self.save_activation)
        target_layer.register_backward_hook(self.save_gradient)

    def save_activation(self, module, input, output):
        self.activations = output.detach()

    def save_gradient(self, module, grad_input, grad_output):
        self.gradients = grad_output[0].detach()

    def generate_cam(self, input_image, target_class=None):
        # Forward pass
        self.model.eval()
        output = self.model(input_image)

        if target_class is None:
            target_class = output.argmax(dim=1).item()

        # Backward pass
        self.model.zero_grad()
        class_score = output[0, target_class]
        class_score.backward()

        # Generate CAM
```

```

        gradients = self.gradients[0] # [C, H, W]
        activations = self.activations[0] # [C, H, W]

        # Global average pooling on gradients
        weights = gradients.mean(dim=(1, 2)) # [C]

        # Weighted combination of activation maps
        cam = torch.zeros(activations.shape[1:], dtype=torch.float32, device=acti
        for i, w in enumerate(weights):
            cam += w * activations[i]

        # Apply ReLU and normalize
        cam = F.relu(cam)
        cam = cam - cam.min()
        cam = cam / cam.max()

        return cam.cpu().numpy(), target_class

def visualize_gradcam(model, image, label, filename, target_layer, class_names, c
    """
    Visualize Grad-CAM for a single image
    """
    # Prepare image
    img_tensor = image.unsqueeze(0).to(device)

    # Generate Grad-CAM
    gradcam = GradCAM(model, target_layer)
    cam, pred_class = gradcam.generate_cam(img_tensor)

    # Resize CAM to match image size
    cam_resized = cv2.resize(cam, (400, 400))

    # Convert tensor image to numpy for visualization
    img_np = image.cpu().permute(1, 2, 0).numpy()
    img_np = (img_np * np.array([0.229, 0.224, 0.225]) + np.array([0.485, 0.456,
    img_np = np.clip(img_np, 0, 1)
    img_np = (img_np * 255).astype(np.uint8)

    # Create heatmap
    heatmap = cv2.applyColorMap(np.uint8(255 * cam_resized), cv2.COLORMAP_JET)
    heatmap = cv2.cvtColor(heatmap, cv2.COLOR_BGR2RGB)

    # Overlay heatmap on image
    overlay = heatmap * 0.4 + img_np * 0.6
    overlay = overlay.astype(np.uint8)

    # Plot
    fig, axes = plt.subplots(1, 3, figsize=(15, 5))

    axes[0].imshow(img_np[:, :, 0], cmap='gray')
    axes[0].set_title(f'Original\nTrue: {class_names[label]}')
    axes[0].axis('off')

    axes[1].imshow(cam_resized, cmap='jet')
    axes[1].set_title('Grad-CAM Heatmap')
    axes[1].axis('off')

```

```

axes[1].axis('off')

axes[2].imshow(overlay)
axes[2].set_title(f'Overlay\nPredicted: {class_names[pred_class]}')
axes[2].axis('off')

plt.tight_layout()
plt.show()

```

```

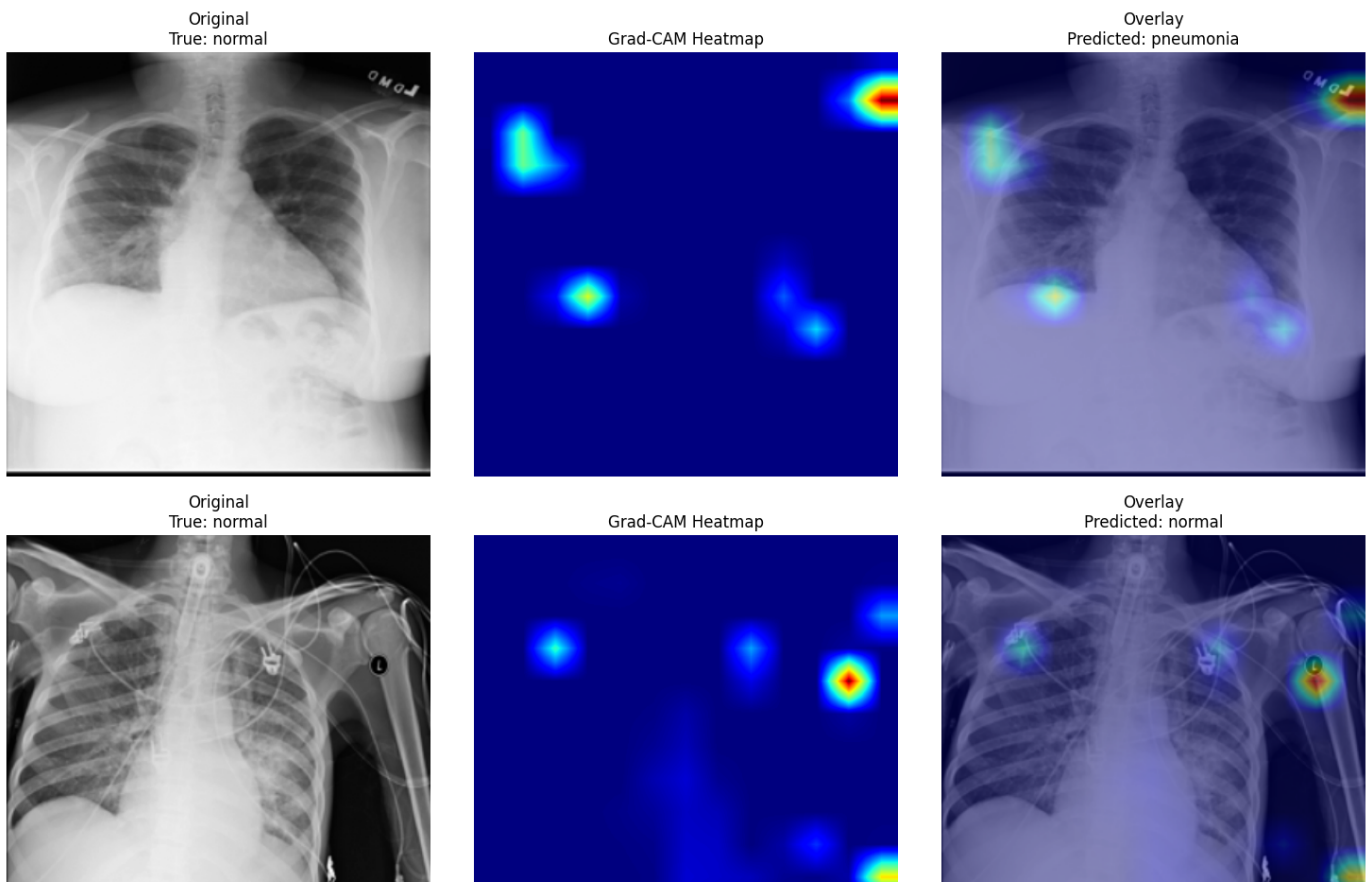
model.eval()
target_layer = model.layer4[-1] # Last layer of ResNet-50

# Get a batch from test loader with reduced workers
test_loader_viz = DataLoader(test_ds, batch_size=8, shuffle=False, num_workers=2,
dataiter = iter(test_loader_viz)
images, labels, filenames = next(dataiter)

# Visualize first 5 images
num_viz = min(5, len(images))
for i in range(num_viz):
    visualize_gradcam(
        model=model,
        image=images[i],
        label=labels[i].item(),
        filename=filenames[i],
        target_layer=target_layer,
        class_names=classes,
        device=device
    )

```

/usr/local/lib/python3.12/dist-packages/torch/nn/modules/module.py:1866: FutureWarning
self._maybe_warn_non_full_backward_hook(args, result, grad_fn)





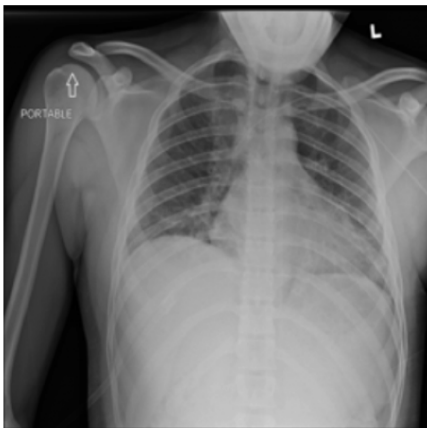
Original
True: normal



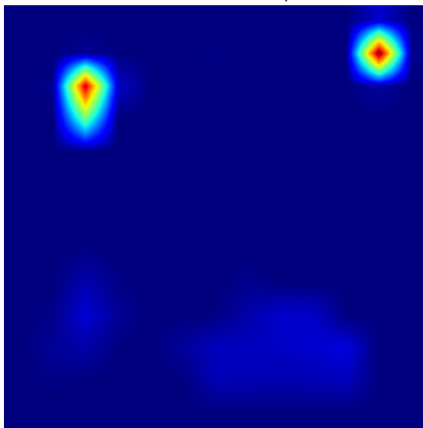
Grad-CAM Heatmap



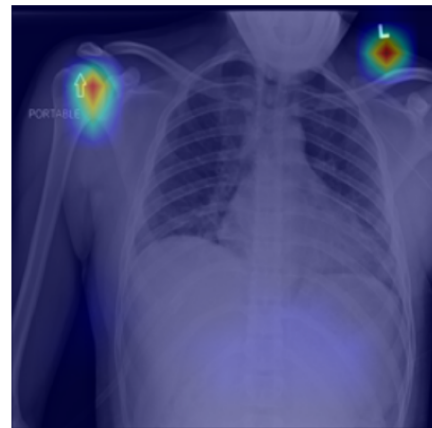
Overlay
Predicted: normal



Original
True: covid



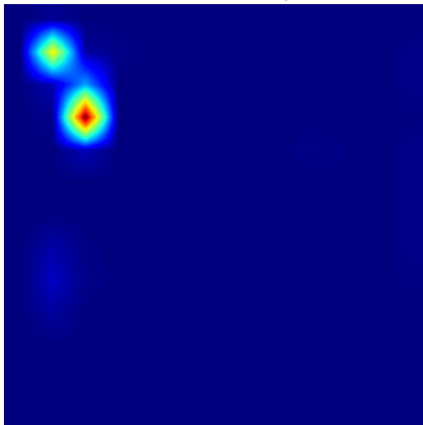
Grad-CAM Heatmap



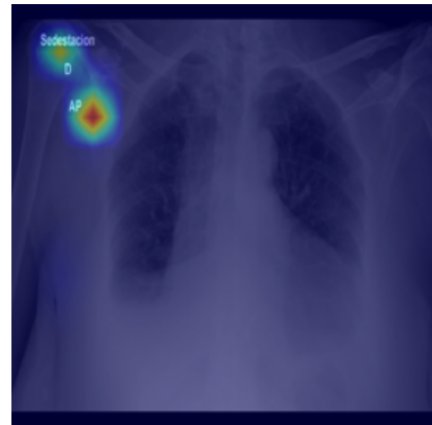
Overlay
Predicted: normal



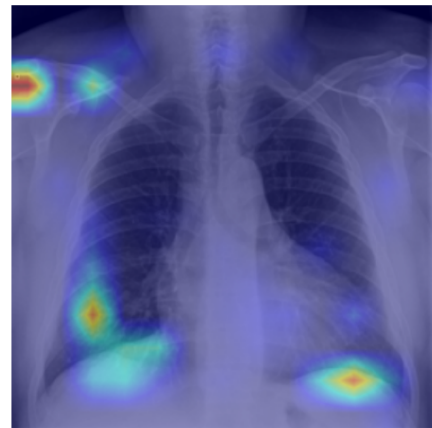
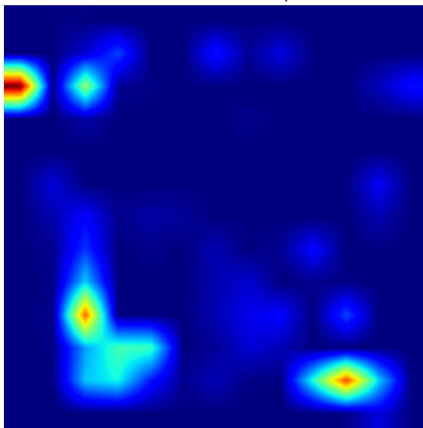
Original
True: covid



Grad-CAM Heatmap



Overlay
Predicted: covid



Right off the bat one can see that there is an obvious confounding variable, which is that some of the chest- xray images contain words or even visible leads within the image that the model devotes a lot of attention to in some cases. It seems to particularly do this in chest x-rays that are labeled normal which might explain why it has the lowest accuracy correctly identifying a normal chest X-ray. That being said these same confounding variables are present in many of the COVID images yet the model devotes more attention to the lungs than the variable itself.

In any other circumstance I, or if I had a team the team, would have to go through all the images, check for confounding variables, and clean them off the image. But this data set is extremely too

large and the task would probably take a month. Without looking at all the grad cam visualizations I can't determine the degree to which the confounding variable is manipulating the results but from the sample available it is clear that the model has the capability to focus primarily on the lungs/chest even when the variable is present and correctly diagnose. That being said if given the chance or more time I would modify the training to penalize the model for focusing on anything but the lungs/chest area as I feel this would provide more accurate metrics on how good the model is. But even then some of the confounding variables such as the leads, which are directly placed on the body may still evade these efforts.

The dataset did include lung mask, which I could also use to train the model on where to focus but I believe that the leads would still confound in that case.